

# The Executable Charter: Architecture

TeamNoClue · Gardenia 2K26 · Problem: Collaborative Research Ecosystem · Focus area C: Fair rewards & governance

Terms that run as code. Every rupee has a receipt. Contribution → Verification → Attribution → Credit / Payment

## 1. Overview

Sponsors post research problems; experts and students form verified teams and work alongside AI agents. Our core idea: the **project charter is structured data that a pure engine runs**, not a PDF. One function reads the charter and a **tamper-evident, hash-chained ledger** of reviewed contributions, and produces every payout with a plain-English **receipt**. Corner cases (someone leaves, the sponsor goes silent, the project turns unpaid) are recorded as ledger events, and the same engine re-runs on them.

At every step the system answers: **who did what** (the ledger), **who could see what** (charter-gated access enforced by the database), and **why each person was paid or credited what they were** (receipts).

## 2. System architecture



Three rules hold the design together: 1. **One write path**. Every state change goes route → lib module → a Postgres function that writes the change *and* its ledger entry in the same transaction. If the ledger append fails, the change rolls back. 2. **The engine is pure**. `computeSplit` has no I/O, clock or randomness, so it is deterministic and fully unit-tested. 3. **One door to the LLM**. Only the agent gateway calls the model, and it enforces scope, ownership and approval.

| COMPONENT                                                            | WHERE                                                       | STATUS         |
|----------------------------------------------------------------------|-------------------------------------------------------------|----------------|
| Login, role-aware dashboard                                          | src/app/login, src/app/dashboard, lib/auth.ts               | Built          |
| 1 Discovery: locked brief, charter accept, leave                     | src/app/projects/[id], lib/charter.ts, lib/access.ts        | Built          |
| 2 AI Scoping & Matching: proposal, ranked candidates, invite         | .../scope, lib/matching.ts, lib/matchingScore.ts            | Built          |
| 3 Workspace: escrow, uploads, integrity check, reviews, agent drafts | .../workspace, lib/work.ts, lib/integrity.ts, lib/drafts.ts | Built          |
| 4 Ledger + Verify                                                    | .../ledger, lib/ledger.ts, lib/ledgerView.ts                | Built          |
| 5 Credit & Payment: waterfall, receipts, accept                      | .../payouts, lib/split.ts, lib/escrow.ts                    | Built          |
| 6 Final record: outcome, human vs AI, credentials, audit trail       | .../record                                                  | Built          |
| Charter engine                                                       | lib/engine/computeSplit.ts, fraction.ts                     | Built + tested |
| AI gateway and four agents                                           | lib/agents/gateway.ts, prompts/                             | Built          |
| Admin console: ledger audit, disputes, demo controls                 | src/app/admin, lib/corner.ts                                | Built          |

**Stack:** Next.js (App Router) + TypeScript, Tailwind CSS, Supabase (Postgres, Auth, RLS), Groq (model from `GROQ_MODEL`, currently `openai/gpt-oss-120b`), Zod, Vitest + PGlite (in-process Postgres for database tests).

## 3. Data model

14 tables. The 10 entities named in the handout, plus `project_briefs` (kept separate so a row-level policy can lock it), `memberships` (which charter version each member accepted), `agent_drafts` (AI output awaiting approval) and `tasks` (how a milestone is split: one human owner per task, closed by a contribution).

| TABLE                     | KEY FIELDS                                                                                                                                                                                       |
|---------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| profiles                  | role (student/expert/sponsor/admin), verified, skills, proven skills, hours per week, affiliations, minor + guardian consent                                                                     |
| projects / project_briefs | sponsor, public summary, sensitivity, declared competitors / confidential brief (locked)                                                                                                         |
| charters                  | project, <b>version</b> , engagement model (funded, stipend, knowledge-sharing, institutional credit), terms                                                                                     |
| memberships               | project, user, role, status (invited/active/inactive/exited), <b>accepted charter version</b> , active fraction                                                                                  |
| milestones / escrows      | criteria, amount, status / amount, status (funded, released, frozen, refunded)                                                                                                                   |
| contributions / reviews   | <b>human owner</b> , agent used, AI share, artefact SHA-256, similarity, <b>version + builds_on</b> (a new version points at the one it replaces; nothing is overwritten) / verdict, impact 0–10 |
| tasks                     | milestone, title, <b>owner</b> , status (todo/in review/done), the contribution that closed it                                                                                                   |
| payouts                   | amount, <b>stored receipt lines</b> , charter version used                                                                                                                                       |
| disputes , agent_drafts   | reason, criterion cited, resolution / AI output pending human approval                                                                                                                           |
| ledger                    | seq, timestamp, project, actor, <b>on_behalf_of</b> , event, payload, prev_hash, hash                                                                                                            |

## 4. Ledger design (tamper evidence)

- **Entry contents:** seq, timestamp, project, actor (a user or `agent:<name>`), `on_behalf_of` (the human owner, required for every agent action), event type, JSON payload, previous hash, own hash.
- **Chaining:** `hash = SHA-256(prev_hash | seq | timestamp | actor | on_behalf_of | event | payload)`, computed in SQL (pgcrypto), so there is one implementation. The first entry's `prev_hash` is 64 zeros.
- **Append:** `ledger_append` takes a transaction-level advisory lock, so the chain stays linear under concurrent writes. It rejects an agent action with no human owner.
- **Append-only:** triggers block `UPDATE`, `DELETE` and `TRUNCATE` ("add a **CORRECTION** entry instead"). Clients have no direct write access; writes happen only through the SQL functions.
- **Verify:** `ledger_verify()` re-computes every hash from entry #1 and returns `ok`, the first broken `seq`, and the count checked. The UI's Verify button calls it and highlights the broken entries.
- **Tamper demo:** someone with raw database access disables the trigger and edits entry #23 (Arjun's AI-assisted contribution). Verify then reports `broken_at = 23`. Production would also anchor the latest hash publicly.

## 5. Charter design and reward logic

The charter is published before anyone applies, accepted **per version** (with a separate acknowledgement of the engagement model), and never changed silently: a new version re-locks the brief until each member re-accepts, and anyone who disagrees can leave under the version they signed, keeping their credit.

**Split (all percentages are charter fields):**

```

distributable = budget – platform fee (10%) – AI compute reserve (5%)
expert total  = 30% × distributable                (shared by experts, scaled by active fraction)
student pool  = distributable – expert total
each student  = 40% × pool ÷ students × active fraction + 60% × pool × weight
weight        = student's reviewed impact ÷ all students' reviewed impact (never commit counts)

```

Arithmetic uses exact fractions (`fraction.ts`). Rounding to whole rupees uses largest remainder, with ties going to the smallest share, so payouts always add up exactly. Pro-rata leftovers are returned as `unallocated`.

**Who is on a milestone's split:** the people who had accepted the charter by the time the milestone was submitted, decided by ledger order. Someone who joins afterwards gets no share of work they were not part of.

**Worked example (funded, ₹1,00,000; reviewed impact Priya 5, Arjun 3, Meera 2):**

| PERSON                                                       | EQUAL PART | WEIGHTED PART | TOTAL          |
|--------------------------------------------------------------|------------|---------------|----------------|
| Dr. Kiran Shetty (expert, 30%)                               | —          | —             | ₹25,500        |
| Priya (weight 0.5)                                           | ₹7,933     | ₹17,850       | ₹25,783        |
| Arjun (0.3)                                                  | ₹7,933     | ₹10,710       | ₹18,643        |
| Meera (0.2)                                                  | ₹7,933     | ₹7,140        | ₹15,074        |
| <b>Total</b> (+ ₹10,000 fee + ₹5,000 AI reserve = ₹1,00,000) |            |               | <b>₹85,000</b> |

**Non-monetary project:** the same engine with a ₹0 budget. Members receive credentials and co-author credit from their ledger record; nothing implies payment. **AI does not earn:** compute is charged to the 5% reserve, credit goes to the human owner, and the AI share is disclosed on the receipt.

## 6. Access control ("who could see what")

Row-level security is enabled on every table (default deny). Key policies: - **Confidential brief:** readable only by the sponsor, or by active members whose accepted charter version equals the latest version. A non-member, an admin, or a member who hasn't re-accepted a new version gets a 403 from the API. Every successful view is written to the ledger as `BRIEF_VIEWED`. - **Ledger, documents, reviews, escrow:** readable by active project members and admins; no direct writes for any client role. Access ends the moment a member exits. - **Payouts:** transparent inside the team. Active members see the whole split and every receipt, read from the `PAYOUT_ISSUED` ledger entries, so anyone on the team can check it. The `payouts` table itself is own-row-only (plus the sponsor), which is all a former member or an outsider can reach. - **Agent drafts:** visible only to the human who owns them, and to admins. - Server code uses the secret key only on the server; the browser only has the publishable key.

## 7. AI agent design

| AGENT              | OWNER                                    | CAN READ                                          | OUTPUT (ALWAYS A DRAFT)                        | WHAT A HUMAN DECIDES                                                |
|--------------------|------------------------------------------|---------------------------------------------------|------------------------------------------------|---------------------------------------------------------------------|
| Scoping            | Sponsor                                  | Summary + brief                                   | Milestones, skills, acceptance criteria (JSON) | Approving the scope                                                 |
| Matching explainer | Whoever runs matching (sponsor or admin) | Score breakdown only                              | One reason per candidate                       | Sending invites                                                     |
| Research / Coding  | The student who runs it                  | This project's summary, brief and their own notes | Draft note or code                             | Editing it and approving it as their contribution                   |
| Review             | Expert                                   | Acceptance criteria + submission record           | Criteria check: met / not met / unclear        | Their own reviews and impact scores, which are what the engine uses |

Gateway rules: the owner must be the sponsor or an active member on the current charter; at most 5 runs per user per minute; document text is wrapped in `<document>` tags and treated as **data, never instructions** (prompt-injection defence, unit-tested); replies are validated with Zod; every call is logged as `AGENT_ACTION` with `on_behalf_of`, tokens and cost; output is saved as a draft and becomes a contribution only when its owner approves it.

**Matching:** hard filters (verified, guardian consent for minors on paid work, no conflict of interest with the sponsor's declared competitors, enough hours, fewer than 2 active projects, at least one required skill) → score out of 100 (skills 40, with skills proven by reviewed work counting more; track record 20; topic fit 15; availability 15; newcomer boost 10) → the LLM writes a reason from the breakdown → top 5 per role → a human invites → the candidate accepts the charter. The filters also apply to the invitation itself, and if the LLM is unavailable the page shows a fixed-wording reason, labelled as such.

## 8. Security mechanism per threat

| THREAT                                 | MECHANISM                                                                                | PROTOTYPE                                     |
|----------------------------------------|------------------------------------------------------------------------------------------|-----------------------------------------------|
| Plagiarism / undisclosed AI            | Similarity score + human review, mandatory AI-use declaration, agent actions auto-logged | Declaration and flag real; score mocked       |
| Theft by insiders                      | Brief only after accepting the charter, every view logged, access ends on exit           | Real (RLS + view log); watermarking not built |
| Theft by sponsor                       | Escrow before work, ledger timestamps as evidence, rejection must cite a criterion       | Escrow simulated                              |
| Credit theft                           | Artefact SHA-256 at submission + tamper-evident ledger                                   | Real                                          |
| Fake accounts / wrong people           | Verified flag, conflict-of-interest filter, human invitation                             | KYC simulated                                 |
| Data leakage via AI / prompt injection | Project-scoped gateway, documents as data, human approval                                | Real (gateway)                                |
| Bait-and-switch terms                  | Versioned charter, re-acceptance, ledger records the version each member accepted        | Real                                          |
| Ledger / payment tampering             | Append-only hash chain, corrections as new entries, Verify                               | Real                                          |

## 9. Corner cases

All five can be triggered live from the admin console's demo controls. Each one calls the same database function a real user's action would, is recorded under the person who would really act, and is marked in the ledger with `demo_control_by`. The engine then re-runs.

| CASE                 | WHAT HAPPENS                                                                                                                              |
|----------------------|-------------------------------------------------------------------------------------------------------------------------------------------|
| Student quits midway | Reviewed credit kept, equal share pro-rata via <code>activeFraction</code> , access revoked at once                                       |
| Sponsor silent       | Auto-accept after the charter's review window; the engine pays out                                                                        |
| Paid becomes unpaid  | New charter version: briefs re-lock, unreleased escrow is refunded, members re-accept or leave with credit; money already paid stays paid |

| CASE                | WHAT HAPPENS                                                                                                            |
|---------------------|-------------------------------------------------------------------------------------------------------------------------|
| Unfair rejection    | A rejection must cite an acceptance criterion; the team can dispute it, which freezes the escrow until an admin decides |
| AI wrote most of it | Credit goes to the human owner; the AI share is printed on the ledger and the receipt                                   |

10. Simplifications (what production would use)

| MOCKED HERE                                      | PRODUCTION                                                                            |
|--------------------------------------------------|---------------------------------------------------------------------------------------|
| Escrow and payments                              | A licensed escrow / payment partner                                                   |
| KYC / identity                                   | A government-ID verification service                                                  |
| Similarity check against one built-in text       | A real plagiarism and AI-detection service                                            |
| Uploaded files are fingerprinted, then discarded | Files kept in private storage with per-member access                                  |
| Seeded users and projects                        | Real sign-ups (no real personal data used)                                            |
| Corner cases triggered on demand by an admin     | Each person acts themselves; inactivity and review windows detected by scheduled jobs |
| Agent cost from a made-up price list             | The provider's real billing, debited from the AI reserve                              |
| One ledger chain                                 | Per-project chains with the latest hash anchored publicly                             |

The README lists every simplification in full.

11. Testing

- `npm test` runs 78 Vitest tests: the engine against the handout's numbers (plus pro-rata exit and zero budget); the schema, seed, RLS policies, hash chain, tamper detection and every state-changing SQL function against an in-process Postgres (PGlite); and the pure rules for access, matching, integrity, the ledger view, the split team and the prompt-injection defence.
- `npm run demo:dryrun` rehearses the whole demo over HTTP against the real Supabase project, including the four agents and every corner case (about 60 checks). It found three bugs that unit tests had missed, all fixed.

12. What changed since Round 2

- Hashing and verification moved **into SQL functions** so every state change and its ledger entry commit together.
- The engine uses **exact fractions** and largest-remainder rounding, so shares always add up to the rupee.
- Database tests run locally on **PGlite**, so the ledger and RLS were tested before touching the cloud project.
- **LLM model:** we planned a Groq-hosted Llama model. Our Groq key is offered no Llama chat model, so the agents run on `openai/gpt-oss-120b`, also hosted by Groq. The gateway is unchanged; the model id is one environment variable.
- **Payouts are transparent inside the team.** The Round 2 design (sheet 5) said the sponsor sees all rows and each member sees their own. We decided a split nobody on the team can check is not a fair split, so active members see all of it.
- **Split team rule added:** only people on the team when a milestone was submitted share in it.
- **Not built:** per-viewer watermarking of the brief, file storage for uploads, and automatic detection of inactivity (nudge at 5 days, inactive at 7). The exits and the sponsor's window are triggered from the demo controls instead.